Mastering XPath for Selenium Test Automation

Author: Amit Dewan

Last updated: 05 January 2023

XPath locators

XPath or **XML Path Language** is a query language for selecting nodes from an **XML document**. XPath also may be used to compute values from the content of an **XML document**.

XPath is the **most complicated** method of identifying elements, but it can find the **location of any element** on a web page, even those without class, name, or id attributes. **XPath produces reliable locators** if it is used correctly.

1. Absolute XPath

Absolute XPaths starts with the root of the html pages and start with **single forward-slash** (/)

Absolute XPaths are not advisable for most of the time due to following reasons

- **Absolute XPaths are lengthier and not readable**
- **Absolute XPaths are brittle when minor structural changes are done to the web pages**

Absolute Xpath is the easiest and direct way to find element but if any changes happen in the path of element, this **XPath** gets failed. It shall be used only when a relative XPath cannot be constructed. (highly unlikely). Hence it **not recommended** to use absolute XPath in Selenium.

Most of the beginners tend use absolute XPaths when they copy them from tools like FireBug, Chropath. Theses XPaths tend to break (brittle) most of the time.

2. Relative XPath

Relative XPaths is used for locating an element from a known element. The element of your choice is referred relative to a known element.

Syntax : Relative XPaths starts with two forward slashes. //

Example : //div[@id='divUsername']/input

Absolute XPaths are faster than the relative XPaths

3. IF there are multiple elements for an XPath?

Selenium will select the first element in the path if there are multiple elements for a given XPath. If you want to specify the location of the element, square brackets with the sequence number shall be used.

Examples :

To select the third input element : **//form/input[3]**

To select the last input element : **//form/input[last()]**

Locating elements by position is discussed further in a separate section

It is important to consider following while choosing a XPath from available options. A good locator is,

- **Unique**
- **Descriptive**
- **Unlikely to change**

4. Locating Elements with Known Attributes

Following syntax can be used for locating elements when at least one attributes are unique and static (not changed).

Syntax : ***//*[@attributeName='value']***

Let's consider the username field

<input id="txtUsername" type="text" name="txtUsername">

Examples :

1. //*[@id='txtUsername']
2. //*[@name='txtUsername']
3. //*[@type='text']

Third XPath is not used even though it is a valid XPath. BUT It is not unique. There might be many elements with attribute type='text'.

5. Locating Elements with Known Visible Text (exact match)

Following syntax is used for locating elements containing some text

<Xpath>

Syntax : ***//elementName[text()='exact text'] or //*[text()='exact text']***

<a target="_blank" href=" https://www.qamentor.com/qa-university/**">University</a>**

Examples :
//a[text()=**University**]
//*[text()=**University**]

The text is case sensitive.
Warning : Locating by the visible text is not advisable if you are testing a multilingual application

6. Locating Elements when part of the visible text is known (partial match)

<Xpath>

Syntax : *//elementName[contains(text(),'part of the text')]*

//elementName[contains(. ,'part of the text')]

or

//[contains(text(),'part of the text')]*

<a target="_blank" href="http://www.orangehrm.com">OrangeHRM, Inc</a>

Examples :

//a[contains(text(),'Orange')]

//a[contains(.,'Orange')]

//[contains(text(),'Orange')]*

Ensure closing brackets are available.

YASH Technologies
More than what you think.

7. Locating elements when starting text is known

<Xpath>

Syntax : ***//elementName[startswith(text(),'starting text')]***

<a target="_blank" href="http://www.orangehrm.com">OrangeHRM, Inc</a>

Example :

//a[startswith(text(),'Orange')]

8. Locating Elements with Dynamic Attributes

Following syntax could be used when a part of the attribute values is changed. We can use the non-changing value for locating the element.

Syntax : ***//elementName[contains(@ attributeName,'part of the value')]***

Example : //a[contains(@href,'orangehrm.com')]

You can also use
//elementName[**startswith**(@attributeName,'starting part of the value')]
And
//*[**startswith**(@attributeName,'starting part of the value')]

The **ends-with** function is part of xpath 2.0 but browsers generally only support 1.0

9. Locating Elements with Multiple Attributes

Sometimes it may not be possible to locate an element with a single attribute. In real world we have similar issue. We cannot locate a person with just first name or last name. We will have to use combination of first name and last name to locate a person without any confusions.

Syntax : ***//*[@attribute1='value1'][@attribute2='value2']...[@attributeN='valueN']***

<input id="btnLogin" class="button" type="submit" value="LOGIN" name="Submit">
//*[@id='btnLogin'][@name='Submit']

Example:
//input[@class='button'][@type='submit'][@value='LOGIN'][@name='Submit']

10. Locating a immediate parent element

The parent axis contains the parent of the context node.

Syntax : ***//parent::* o r //parent::elementName***

```
<ul id="menu45" name="menubutton46" >
      <li name="Folder" class="menu-item">
                  <a id="google" href="http://google.com">Google</a>
      </li>
      <li name="bookmark" class="menu-item">

 <a id="yahoo" href="http://yahoo.com">Yahoo</a>
      </li>
</ul>
```

Example:
XPath of the known element **//a[@id="yahoo"]**
Now we can locate the parent '**li**' element with respect to the '**a**' element
//a[@id="yahoo"]/parent::li
Also
//a[@id="yahoo"]/parent::*
This can also be written as **//a[@id="yahoo"]/..**

There can be only one (immediate) parent to a context (known) element. Hence specifying the element name is optional.

11. Locating a child element

The child axis contains the children of the context node

Syntax: //<xpathOfKnownElement>/child::<elementName> or //<xpathOfKnownElement>/child::*

```
<ul id="menu45" name="menubutton46" >
    <li name="Folder" class="menu-item">
            <a id="google" href="http://google.com">Google</a>
    </li>
    <li name="bookmark" class="menu-item">
            <a id="yahoo" href="http://yahoo.com">Yahoo</a>
    </li>
</ul>
```

Example:
Xpath of the known element **//ul[contains(@name,'menubutton')]**

Xpath to the child element is **//ul[contains(@name,'menubutton')]/child::li[@name='Folder']**

You can omit **child::** from the XPath. **//ul[contains(@name,'menubutton')]/li[@name='Folder']**

12. Locating ancestors of a known element

The ancestor axis contains the ancestors of the known element ; ancestor axis consists of the parent of known element and the parent's parent so on..

Syntax: ***//<xpathOfKnownElement>/ancestor::<elementName> or //<xpathOfKnownElement>/ancestor::****

```
<ul id="menu45" name="menubutton46" >
    <li name="Folder" class="menu-item">
            <a id="google" href="http://google.com">Google</a>
    </li>
    <li name="bookmark" class="menu-item">
            <a id="yahoo" href="http://yahoo.com">Yahoo</a>
    </li>
</ul>
```

Example:

//a[@id="yahoo"]/ancestor::ul

Or

//a[@id="yahoo"]/ancestor::li

Or

//a[@id="yahoo"]/ancestor::* **(retrieves all ancestors)**

This will return all ancestor elements starting from html

13. Locating following elements

Keyword following can be used for locating an element anywhere below the tree with respect to the known element.

Syntax: ***//<xpathOfKnownElement>/following::<elementName> or //<xpathOfKnownElement>/following::***

```
<div name="parallel">
First name: <input type="text" name="FirstName" value="karan"><br>
<input type="submit" value="Submit">
<select>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
        <option value="mercedes">Mercedes</option>
        <option value="audi">Audi</option>
</select>
</div>
<input type="radio" name="gender" value="male" checked> Male<br>
  <input type="radio" name="gender" value="female"> Female<br>
  <input type="radio" name="gender" value="other"> Other  <br>
```

Example:

//div[@name='parallel']/following::input[@value='male']

Any descendant elements are excluded

14. Locating preceding element

Keyword **preceding** is used for locating any element above the tree with respect to a known (xpath) element.

Syntax: ***//<xpathOfKnownElement>/preceding::<elementName> or //<xpathOfKnownElement>/***

```
<div name="parallel">
First name: <input type="text" name="FirstName" value="karan"><br>
<input type="submit" value="Submit">
<select>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="mercedes">Mercedes</option>
  <option value="audi">Audi</option>
</select>
</div>
<input type="radio" name="gender" value="male" checked> Male<br>
  <input type="radio" name="gender" value="female"> Female<br>
  <input type="radio" name="gender" value="other"> Other  <br>
```

Example:

//input[@value='male']/preceding::select

Any ancestors are excluded

15. Single slash (/) and Double slash(//) in XPath

```
<div class="form-buttons">
    <button id="login" type="submit" class="btn-primary"> Next</button>
    <div class="form-input">
        <input id="login-email" type="text" name="login" class="inp-width">
    </div>
</div>
```

Example with single-slash which means moving to child element at immediate level:
Here we are finding xpath locator for button element.
//div[@class='form-buttons']/button[@id='login']

Example with double-slash which means moving to child element at any level:
Here we are finding xpath locator of input element with reference to first div
//div[@class='form-buttons']//input[@id='login-email']

16. (.) operator for finding parent using property of child

```
<ul name="products">
      <li>rice</li>
     <li>banana</li>
     <li>Milk</li>
</ul>
<ul name="products">
    <li>Samsung</li>
    <li>LG</li>
    <li>Sony</li>
</ul>
```

Example with dot operator which means using child element property to find parent element
Here we are finding xpath locator for first **ul** element.

//ul[@name='products' and ./li[text()='Milk']]

17. Using(.) in place of text() in xpath

```
<li aria-label="Create a new Box Note" class="menu-item" role="menuitem" tabindex="-1">
        <span>Box Note</span>
</li>
```

Then:

xpath= //li[.='Box Note'] returns the **<li>** element

whereas

xpath= //li[text()='Box Note'] does not return anything because text is within the **** element and not **<li>** element

and also

xpath= //span[text()='Box Note'] and **xpath= //span[.='Box Note']** both are correct because **** element contains the text.

Quick XPath summary

XPath	Description
//*[@attribute='value']	Locating element with known attribute.
//element[@attribute='value']	Locating element with known element and attribute.
//element[text()='exact text']	Locating Elements with Known Visible Text (exact match)
//*[text()='exact text']	Locating Elements with Known Visible Text (exact match)
/elementName[contains(text(),'part of the text')]	Locating Elements when part of the visible text is known (partial match)
//elementName[contains(.,'part of the text')]	Locating Elements when part of the visible text is known (partial match)
//*[contains(text(),'part of the text')]	Locating Elements when part of the visible text is known (partial match)
//element[starts-with(text(),'starting text')]	Locating elements when starting text is known
//elementName[contains(@attributeName,'part of the value')]	Locating Elements when part of the attribute value is known (partial match)

Quick XPath summary

//*[contains(@attributeName,'part of the value')]	**Locating Elements when part of the attribute value is known (partial match)**
//*[attribute1='value1'][attribute2='value2']...[attributeN='valu eN']	Locating elements with multiiple attributes.
//<knownXpath>/parent::*	Locate the parent element with respect to the child element
//<knownXpath>/parent::elementName	Locate the parent element with respect to the child element
//<xpathOfKnownElement>/ancestor::<e lementName>	Locator for returning a parent element of known element
//<xpathOfKnownElement>/ancestor::*	Locator for returning all parent elements of known element starting from html
//<xpathOfKnownElement>/following::<elementName>	Locator for locating an element anywhere below the tree with respect to the known element.
//<xpathOfKnownElement>/following::*	Locator for locating all elements anywhere below the tree with respect to the known element.
//<xpathOfKnownElement>/preceding::<e lementName> //<xpathOfKnownElement>/preceding::*	Locator for locating an element anywhere above the tree with respect to the known element.
Xpath[position] , Xpath[last()] , Xpath[last()n]	Locating an element explicitly with index when there are many elements